

Data Transfer and Addressing

Load/Store architecture, addressing modes, memory alignment rules, and array/pointer traversal in AArch64

Topic 6 Outline

Topic Objective

Analysis of AArch64 load-store memory architecture, memory addressing modes, alignment constraints, and data structure mapping for arrays and pointers.

- **Section 1:** The Load-Store Architecture Paradigm
- **Section 2:** Load and Store Instructions (`ldr`, `str`)
- **Section 3:** AArch64 Memory Addressing Modes
- **Section 4:** Memory Access, Alignment, and Pair Transfers
- **Section 5:** Traversing Arrays and Pointers
- **Section 6:** Summary and Worked Self-Test Exercises

Bridging Processor Registers and Memory

The Memory Wall Challenge

CPU execution speeds outpace main memory bandwidth; explicit data transfer instructions manage movement between high-speed registers and byte-addressed storage.

- Arithmetic instructions operate exclusively on registers, never directly on memory.
- Programs require specialized transfer instructions to read from and write to memory.
- Efficient memory access depends on correct addressing mode selection and natural alignment.

The Load-Store Paradigm Architecture

Load-Store Architecture Principles

AArch64 follows a strict RISC load-store design philosophy:

- **Separation of Concerns:** Computation instructions (add, sub) only process register operands. Memory instructions (ldr, str) only move data.
- **No In-Memory Arithmetic:** An instruction cannot simultaneously fetch a value from memory, add to it, and store it back.
- **Simplified Datapath:** Keeps hardware control logic streamlined, predictable, and optimized for pipelining.

Implication for Compilers

C statements like `a = b + c;` compile into three distinct steps: load b, load c, add them in registers, and store the sum into a.

Memory Addresses as Register Values

Memory locations are targeted using 64-bit virtual addresses held in general-purpose registers:

- The base address of a variable, array, or data structure is computed or loaded into an X register (e.g., x1).
- Load/store instructions treat this register as a pointer referencing the target byte location.
- Arithmetic operations can manipulate pointers dynamically (e.g., advancing through indices).

Pointer Register Rule

Only 64-bit general-purpose registers (x0–x30) can serve as base address pointers in AArch64.

Load and Store Instructions

Loading Data: The `ldr` Instruction

The Load Register (`ldr`) instruction copies data from memory into a target register:

- **Syntax Format:** `ldr destination_reg, [base_register]`
- **64-bit Transfer:** `ldr x0, [x1]` loads 8 bytes from the address in `x1` into `x0`.
- **32-bit Transfer:** `ldr w0, [x1]` loads 4 bytes into `w0`, zero-extending the upper 32 bits of `x0`.

Data Sizing Suffixes

Sub-word loads are supported via explicit qualifiers like `ldrb` (byte) and `ldrh` (halfword), with sign-extended variants (`ldrshb`, `ldrshh`).

Storing Data: The `str` Instruction

The Store Register (`str`) instruction copies data from a register into memory:

- **Syntax Format:** `str source_reg, [base_register]`
- **64-bit Transfer:** `str x0, [x1]` writes the 8-byte contents of `x0` to the memory address held in `x1`.
- **32-bit Transfer:** `str w0, [x1]` writes the lower 4 bytes of `w0` to memory.

Sub-Word Stores

Corresponding store variants `strb` (store byte) and `strh` (store halfword) allow granular writing of lower register bytes to memory destinations.

Precision Data Sizing and Sign Extension

AArch64 load instructions allow precise control over data width and extension behavior:

Instruction	Bytes Loaded	Extension Behavior
<code>ldrb w0, [x1]</code>	1 Byte (8 bits)	Zero-extended to 32 bits in <code>w0</code>
<code>ldrsb x0, [x1]</code>	1 Byte (8 bits)	Sign-extended to 64 bits in <code>x0</code>
<code>ldrh w0, [x1]</code>	2 Bytes (16 bits)	Zero-extended to 32 bits in <code>w0</code>
<code>ldrsh x0, [x1]</code>	2 Bytes (16 bits)	Sign-extended to 64 bits in <code>x0</code>

Sign Extension Importance

When loading signed 8-bit or 16-bit integers into larger registers, using sign-extended variants (`ldrsb/ldrsh`) preserves negative values correctly.

AArch64 Modes

Memory

Addressing

Addressing Mode Classification

An addressing mode defines how the CPU calculates the target memory address for a load or store instruction:

- **Base Register (Indirect):** Address is taken directly from the base register.
- **Immediate Offset:** Adds a constant offset to the base register without modifying the base.
- **Register Offset:** Adds a scaling or offset register to the base address.
- **Pre-Indexed:** Applies an offset, updates the base register, and accesses memory.
- **Post-Indexed:** Accesses memory using the base address, then updates the base register.

Base Register (Indirect) Addressing

Syntax and Semantics

```
ldr x0, [x1]
```

The memory address is taken strictly from the contents of the base register x1. The base register remains unchanged after execution.

- **Use Case:** Accessing a variable or dereferencing a pointer whose exact address is already loaded in a register.
- Analogous to pointer dereferencing in C: `*ptr`.

Immediate Offset Addressing

Syntax and Semantics

```
ldr x0, [x1, #16]
```

Computes the target address by adding an immediate constant offset (+#16) to the base register x1. The base register x1 is **not** modified.

- **Use Case:** Accessing fields within a structure or fixed offsets in a local stack frame.
- Immediate range rules depend on transfer size (must align with data width constraints).

Pre-Indexed Addressing Mode

Syntax and Semantics

```
ldr x0, [x1, #8] !
```

The immediate offset (+#8) is added to base register x1. This new calculated address is used for the memory load, **and** is written back into x1.

- Identified by the trailing exclamation mark (!).
- **Use Case:** Updating a pointer before accessing the next sequential memory item.

Post-Indexed Addressing Mode

Syntax and Semantics

```
ldr x0, [x1], #8
```

Data is loaded from the unmodified address in x1. After the memory access completes, the offset (+#8) is added to x1, updating the pointer register.

- Identified by placing the offset outside the square brackets.
- **Use Case:** Fetching current array elements and automatically advancing the pointer for the next iteration.

Register Offset and Scaled Addressing

AArch64 allows an index register to offset the base pointer, optionally scaled by data size:

- **Syntax:** `ldr x0, [x1, x2]` → Address is $x1 + x2$.
- **Scaled Register Offset:** `ldr x0, [x1, x2, lsl #3]` → Address is $x1 + (x2 \times 8)$.

Array Indexing Power

Scaling shifts the index register by the exact byte width of the data item ($2^3 = 8$ bytes for doublewords), mapping C array subscripts directly into hardware addressing.

Addressing Mode Summary

Mode	AArch64 Assembly	Address	Base Register
Offset	<code>ldr x0, [x1, x2]</code>	$x1 + x2$	Unchanged
Pre-index	<code>ldr x0, [x1, x2] !</code>	$x1 + x2$	$x1 = x1 + x2$
Post-index	<code>ldr x0, [x1], x2</code>	$x1$	$x1 = x1 + x2$

Note on Register vs. Immediate

Offset and index quantities can be specified as general-purpose registers (as shown above) or compatible immediate byte constants (e.g., #8).

Memory Access, Alignment, and Pair Transfers

Memory Alignment Rules in AArch64

Data alignment dictates how memory addresses map to data sizes:

- **Natural Alignment:** An n -byte data item must be stored at an address divisible by n .
 - Doublewords (8 bytes) require addresses divisible by 8.
 - Words (4 bytes) require addresses divisible by 4.
 - Halfwords (2 bytes) require addresses divisible by 2.
- **Strict Enforcement:** Unlike x86, unaligned general-purpose memory accesses in AArch64 can trigger hardware alignment fault exceptions (unless explicitly permitted by specific architectural modes).

Load and Store Pair Instructions (ldp / stp)

AArch64 provides specialized instructions to transfer **two registers simultaneously**:

- **Load Pair (ldp)**: `ldp x0, x1, [x2]` loads two 64-bit registers from consecutive memory locations starting at `x2`.
- **Store Pair (stp)**: `stp x0, x1, [x2]` stores two 64-bit registers consecutively to memory.

Performance Efficiency

Pair instructions utilize wide memory data buses, moving 16 bytes in a single instruction cycle and reducing overall instruction count.

Stack Operations via Pair Transfers

Stack management relies heavily on pre-index and post-index pair instructions:

Pushing to the Stack (Pre-Index)

```
stp x29, x30, [sp, #-16]!
```

Decrements stack pointer (sp) by 16 bytes, then stores frame pointer (x29) and link register (x30).

Popping from the Stack (Post-Index)

```
ldp x29, x30, [sp], #16
```

Loads registers from current sp, then increments stack pointer by 16 bytes, restoring stack balance.

Traversing Arrays and Pointers

Mapping C Arrays to AArch64 Assembly

Arrays consist of contiguous memory locations holding elements of identical size:

- Let `x1` hold the base address of an integer array (`int A[]`, where each element is 4 bytes).
- Accessing element `A[i]` requires computing the effective address:
 $\text{Base} + (\text{Index} \times \text{Element Size})$.
- Assembly implementations utilize scaled register offsets or pointer arithmetic loops.

Example Array Access

`ldr w0, [x1, x2, lsl #2]` loads `A[i]` into `w0`, where `x2` holds index `i` scaled by $2^2 = 4$.

Array Traversal via Pointer Increments

Loops often traverse arrays by incrementing pointers directly:

- Initializing base pointer `x1` to the start of an array.
- Using post-indexed loads to fetch data and advance the pointer in one instruction:

```
ldr x0, [x1], #8
```

- Eliminates the overhead of maintaining a separate loop index register.

Loop Termination

Loops compare the current pointer against an end-of-array sentinel address using comparison (`cmp`) and conditional branch instructions.

Accessing Structures and Records

Structures group heterogeneous data fields at fixed byte offsets from a base pointer:

- Suppose a struct has fields at offsets 0 (ID), 4 (Age), and 8 (Score).
- Base address of struct instance held in pointer register x1.
- Immediate offset addressing accesses individual members without modifying the base pointer:
 - `ldr w0, [x1, #0]` (Load ID)
 - `ldr w2, [x1, #4]` (Load Age)
 - `ldr x3, [x1, #8]` (Load Score)

Summary & Self-Test

Topic 6 Comprehensive Summary

- **Load-Store Paradigm:** Memory access is strictly restricted to explicit `ldr` and `str` instructions; arithmetic occurs exclusively in registers.
- **Addressing Modes:** Supports base indirect, immediate offset, register offset, pre-indexed, and post-indexed pointer manipulation.
- **Alignment Rules:** Natural alignment is strictly enforced to prevent hardware alignment faults.
- **Pair Transfers:** `ldp` and `stp` optimize multi-register memory access and stack frame management.
- **Data Structures:** Scaled offsets and post-indexing enable efficient traversal of arrays and structures.

Self-Test: Questions 1 & 2

Question 1: Load-Store Architecture

Why does AArch64 enforce a strict load-store architecture, prohibiting arithmetic instructions from directly operating on memory operands?

Question 2: Immediate vs. Indexed Addressing

What is the primary operational difference between immediate offset addressing (`[x1, #8]`) and pre-indexed addressing (`[x1, #8]!`)?

Self-Test: Solutions 1 & 2

Solution 1

Separating load/store operations from computation simplifies hardware datapath design, enables uniform instruction lengths, keeps pipelines predictable, and avoids complex multi-cycle memory-access execution cycles.

Solution 2

Immediate offset adds the constant to the base register solely to calculate the target address, leaving the base register unchanged. Pre-indexed addressing calculates the address **and writes the updated address back** into the base register.

Self-Test: Questions 3 & 4

Question 3: Post-Indexed Array Traversal

Explain how the instruction `ldr x0, [x1], #8` executes step-by-step and why it is useful for array traversal.

Question 4: Memory Alignment

What happens when an unaligned 64-bit doubleword load is executed on standard AArch64 hardware?

Self-Test: Solutions 3 & 4

Solution 3

It loads data from the current address in x1 into x0, and then increments x1 by 8 bytes. This automates pointer advancement, making sequential array traversal concise and efficient.

Solution 4

Unaligned accesses violate natural alignment rules and typically trigger a hardware alignment fault exception, crashing the program or invoking an OS trap handler.

Self-Test: Questions 5 & 6

Question 5: Pair Transfers on the Stack

How do `stp` and `ldp` manage stack pointer updates during function entry and exit?

Question 6: Scaled Register Addressing

In the instruction `ldr x0, [x1, x2, lsl #3]`, what role does the `lsl #3` modifier play in indexing an array of 64-bit integers?

Self-Test: Solutions 5 & 6

Solution 5

Using pre-indexed store pair (`stp ... [sp, #-16]!`), the stack pointer is decremented and registers are saved in one atomic step. Post-indexed load pair (`ldp ... [sp], #16`) restores registers and re-increments the stack pointer.

Solution 6

The `lsl #3` scales the index in `x2` by multiplying it by $2^3 = 8$ (the byte size of a 64-bit doubleword), correctly translating logical array indices into byte offsets.

Review: Addressing Mode Summary Table

Addressing Mode	Assembly Syntax Example	Base Register State After Access
Base Register	<code>ldr x0, [x1]</code>	Unchanged
Immediate Offset	<code>ldr x0, [x1, #16]</code>	Unchanged
Pre-Indexed	<code>ldr x0, [x1, #8]!</code>	Updated ($\text{Base} + 8$)
Post-Indexed	<code>ldr x0, [x1], #8</code>	Updated ($\text{Base} + 8$)

Wrap-Up & Transition to Topic 7

Moving Forward to Topic 7

Now that we have mastered memory data transfer, addressing modes, and alignment, our next session will explore:

- **Control Flow & Branching:** Unconditional and conditional branches in AArch64.
- **Subroutine Linkage:** Function calls (`bl`), return linkage (`ret`), and ABI parameter passing conventions.

End of Topic 6 — Questions & Discussion

Topic 6 Source Notes

Source Reference Verification

- Load-store mechanics and offset calculations follow standard ARMv8/ARMv9 AArch64 ISA architecture guidelines.
- Stack frame management conforms to standard Procedure Call Standard for the ARM 64-bit Architecture (AAPCS64).